In [2]:
```
#impot libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sqlite3
import seaborn as sns
```

In [4]:
```
#import database
conn = sqlite3.connect('customer_churn.db')

sql_query = """ SELECT name
            FROM sqlite_master
            WHERE type='table';"""

tables = pd.read_sql(sql_query, conn)

#create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}",conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

In [5]:
```
#print table names and columns names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    #Get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

#Close connection
conn.close()
```

```
Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan
_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charge
s', 'cltv', 'churn_score']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

In [6]:
```
# Data Cleaning
```

In [7]:
```
df_db_customer.head()
```

Out[7]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

In [8]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   customerid  21 non-null     str
 1   name        21 non-null     str
 2   country     18 non-null     str
 3   state       21 non-null     str
 4   gender      21 non-null     str
 5   dob         21 non-null     str
 6   interests   4 non-null      str
 7   pincode     0 non-null      object
dtypes: object(1), str(7)
memory usage: 2.5+ KB
```

In [9]:
```
df_db_customer.isna()
```

Out[9]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

False	False	False	False	False	False	False	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	False	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	False	True
False	False	True	False	False	False	True	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	True	True
False	False	True	False	False	False	True	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	True	True
False	False	True	False	False	False	True	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	False	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	True	True
False	False	False	False	False	False	True	True

In [10]:
```
df_db_customer.isna().sum()
```

Out[10]:
```
customerid     0
name           0
country        3
state          0
gender         0
dob            0
interests     17
pincode       21
dtype: int64
```

In [11]:
```
# 1- Drop columns - intrest and pincode
# 2- rename columns
# 3- change datatype of -dob
# 4- fix missing values
# 5- gender fix(data standardization)
```

In [12]:
```
# rename column
df_db_customer.rename(columns = {'name':'customer_name'},inplace=True)
```

In [13]:
```
df_db_customer
```

Out[13]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	None
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	None
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	None
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	None
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	None
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	None
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	None
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	None
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	None
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	None
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	None

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [14]:
```
# drop columns
df_db_customer.drop(df_db_customer.columns[-2:],axis=1,inplace=True)
```

In [15]:
```
df_db_customer.head()
```

Out[15]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	

In [16]:
```
# chnage datatype
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        18 non-null     str
 3   state          21 non-null     str
 4   gender         21 non-null     str
 5   dob            21 non-null     str
dtypes: str(6)
memory usage: 2.2 KB
```

In [17]:
```
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

In [18]:
```
df_db_customer
```

Out[18]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody> </tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11
0020-INWCK	parvati	India	Delhi	Female	1986-02-28
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06

In [19]:
```
# data standardization - gender
df_db_customer['gender']
```

Out[19]:
```
0       Male
1       Male
2     Female
3       Male
4     Female
5      Women
6     Female
7       Male
8      Women
9       Male
10       Men
11    Female
12    Female
13    Female
14       Men
15    Female
16    Female
17    Female
18      Male
19       Men
20     Women
Name: gender, dtype: str
```

In [20]:
```
df_db_customer['gender'].unique()
```

Out[20]:
```
<ArrowStringArray>
['Male', 'Female', 'Women', 'Men']
Length: 4, dtype: str
```

In [21]:
```
df_db_customer['gender'].replace({'Men':'Male',
                                  'Women':'Female'},inplace=True)
```

```
C:\Users\Admin\AppData\Local\Temp\ipykernel_23436\544754834.py:1: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html
  df_db_customer['gender'].replace({'Men':'Male',
```

Out[21]:
```
0       Male
1       Male
2     Female
3       Male
4     Female
5     Female
6     Female
7       Male
8     Female
9       Male
10      Male
11    Female
12    Female
13    Female
14      Male
15    Female
16    Female
17    Female
18      Male
19      Male
20    Female
Name: gender, dtype: str
```

In [22]:
```
# fix missing values
df_db_customer['country'].isna()
```

Out[22]:
```
0     False
1     False
2     False
3     False
4     False
5      True
6     False
7     False
8      True
9     False
10    False
11    False
12     True
13    False
14    False
15    False
16    False
17    False
18    False
19    False
20    False
Name: country, dtype: bool
```

In [23]:
```
# country and state - unique value pair
state_country_mapping = df_db_customer.dropna(subset = ['country']).set_index('stat

df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['state'
```

In [24]:
```
df_db_customer['country'].isna()
```

Out[24]:
```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9     False
10    False
11    False
12    False
13    False
14    False
15    False
16    False
17    False
18    False
19    False
20    False
Name: country, dtype: bool
```

In [25]:
```
df_db_subscription.head()
```

Out[25]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaN	NaN	13.99	627	12
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaN	NaN	6.99	210	34
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaN	NaN	22.99	1725	8
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

In [26]:
```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     str
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     str
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      str
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 3.1 KB
```

In [27]:
```
df_db_subscription.isna().sum()
```

Out[27]:
```
customerid                0
subscription_start_date   0
subscription_type         0
renewal_date              0
plan_type                 0
contract_type             0
cancellation_date         15
cancellation_reason       15
monthly_charges           0
cltv                      0
churn_score               0
dtype: int64
```

In [28]:
```
date_col = ['subscription_start_date','renewal_date','cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     datetime64[us]
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     datetime64[us]
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      datetime64[us]
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 2.7 KB
```

In [29]:
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      str
dtypes: int64(1), object(1), str(4)
memory usage: 895.0+ bytes
```

In [30]:
```
df_db_support.drop(columns=['col_1','comment'],inplace=True)
```

In [31]:
```
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])

df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      datetime64[us]
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 519.0 bytes
```

In [32]:
```
# Feature Engineering and Data Analysis
```

In [33]:
```
df_db_subscription.head()
```

Out[33]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

In [34]:
```
# create a new column using existing column - churn flag
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date']
```

In [35]:
```
df_db_subscription
```

Out[35]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	0
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	0
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	0
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1
0013-MHZWF	2022-06-18	Paid	2025-06-18	Standard	Annual	NaT	NaN	17.99	720	22	0
0013-SMEOE	2021-09-30	Refferal	2024-09-30	Basic	Monthly	2024-11-15	Not enough content	8.99	230	79	1
0014-BMAQU	2020-02-14	Organic	2025-02-14	Premium	Annual	NaT	NaN	22.99	1840	5	0
0015-UOCOJ	2023-07-22	Organic	2024-07-22	Standard	Monthly	NaT	NaN	13.99	240	34	0
0016-QLJIS	2022-04-03	Organic	2025-04-03	Basic	Annual	NaT	NaN	6.99	335	41	0
0017-DINOC	2021-12-01	Organic	2025-12-01	Premium	Annual	NaT	NaN	22.99	1610	14	0
0017-IUDMW	2023-03-17	Refferal	2024-03-17	Standard	Monthly	2024-05-01	Poor streaming quality	7.99	270	83	1
0018-NYROU	2020-10-09	Organic	2025-10-09	Standard	Annual	NaT	NaN	13.99	980	19	0
0019-EFAEP	2022-08-25	Refferal	2024-08-25	Basic	Monthly	2024-10-31	Switched to competitor	12.99	160	76	1

0019-GFNTW	2019-11-11	Paid	2025-11-11	Premium	Annual	NaT	NaN	92.99	2185	3	0
0020-INWCK	2023-05-06	Organic	2025-05-06	Standard	Monthly	NaT	NaN	13.99	640	58	0
0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	Annual	NaT	NaN	20.99	550	62	0
0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	Annual	NaT	NaN	13.99	840	27	0
0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	Monthly	2024-09-14	Forgot to cancel trial	16.99	42	99	1
0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	Annual	NaT	NaN	22.99	1955	7	0
0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	Monthly	NaT	NaN	13.99	790	47	0

In [36]:
```
#merge
df = (df_db_subscription
            .merge(df_db_customer, on = 'customerid', how = 'left')
            .merge(df_db_support, on = 'customerid', how = 'left') )
```

In [37]:
```
df.shape
```

Out[37]:
```
(23, 20)
```

In [38]:
```
df.head()
```

Out[38]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	0	k
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1	r
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1	r
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	0	
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	0	n

In [39]:
```
df_db_subscription['customerid'].unique()
```

Out[39]:
```
<ArrowStringArray>
['0002-ORFBO', '0003-MKNFE', '0004-TLHLJ', '0011-IGKFF', '0013-EXCHZ',
 '0013-MHZWF', '0013-SMEOE', '0014-BMAQU', '0015-UOCOJ', '0016-QLJIS',
 '0017-DINOC', '0017-IUDMW', '0018-NYROU', '0019-EFAEP', '0019-GFNTW',
 '0020-INWCK', '0020-JDNXP', '0021-IKXGC', '0022-TCJCI', '0023-HGHWL',
 '0023-UYUPN']
Length: 21, dtype: str
```

In [40]:
```
df_db_customer['customerid'].unique()
```

Out[40]:
```
<ArrowStringArray>
['0002-ORFBO', '0003-MKNFE', '0004-TLHLJ', '0011-IGKFF', '0013-EXCHZ',
 '0013-MHZWF', '0013-SMEOE', '0014-BMAQU', '0015-UOCOJ', '0016-QLJIS',
 '0017-DINOC', '0017-IUDMW', '0018-NYROU', '0019-EFAEP', '0019-GFNTW',
 '0020-INWCK', '0020-JDNXP', '0021-IKXGC', '0022-TCJCI', '0023-HGHWL',
 '0023-UYUPN']
Length: 21, dtype: str
```

In [41]:
```
df_db_support['customerid'].unique()
```

Out[41]:
```
<ArrowStringArray>
['0003-MKNFE', '0013-EXCHZ', '0013-MHZWF', '0013-SMEOE', '0017-IUDMW',
 '0019-EFAEP', '0022-TCJCI']
Length: 7, dtype: str
```

In [42]:
```
df_db_support
```

Out[42]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody> <tbody> </tbody>

0003-MKNFE	2024-08-28	N	60	
0003-MKNFE	2024-08-28	Y	10	
0013-EXCHZ	2024-01-20	Y	20	
0013-MHZWF	2025-03-18	N	90	
0013-SMEOE	2024-11-01	N	30	
0017-IUDMW	2024-04-10	Y	25	
0019-EFAEP	2024-09-27	Y	30	
0022-TCJCI	2024-09-13	Y	10	
0022-TCJCI	2024-09-14	N	90	

In [43]:
```
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'
```

In [44]:
```
df_db_support
```

Out[44]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody> <tbody></tbody>

	0003-MKNFE	2024-08-28	N	60	2
	0003-MKNFE	2024-08-28	Y	10	2
	0013-EXCHZ	2024-01-20	Y	20	1
	0013-MHZWF	2025-03-18	N	90	1
	0013-SMEOE	2024-11-01	N	30	1
	0017-IUDMW	2024-04-10	Y	25	1
	0019-EFAEP	2024-09-27	Y	30	1
	0022-TCJCI	2024-09-13	Y	10	2
	0022-TCJCI	2024-09-14	N	90	2

In [45]:
```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('custom
```

In [46]:
```
df_db_support
```

Out[46]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> </thead> <tbody> <th>2</th> <th>5</th> <th>1</th> <th>8</th> <th>6</th> <th>4</th> <th>3</th> </tbody> <tbody></tbody>

	0013-EXCHZ	2024-01-20	Y	20	1
	0017-IUDMW	2024-04-10	Y	25	1
	0003-MKNFE	2024-08-28	Y	10	2
	0022-TCJCI	2024-09-14	N	90	2
	0019-EFAEP	2024-09-27	Y	30	1
	0013-SMEOE	2024-11-01	N	30	1
	0013-MHZWF	2025-03-18	N	90	1

In [47]:
```
#merge df
df = (df_db_subscription
            .merge(df_db_customer, on = 'customerid', how = 'left')
            .merge(df_db_support, on = 'customerid', how = 'left') )
```

In [48]:
```
df.shape
```

Out[48]:
```
(21, 21)
```

In [49]:
```
df.to_csv('exported_churn_data.csv', index=False)
```

In [50]:
```
#data analysis
```

In [51]:
```
#1 -churndate
```

In [52]:
```
df.columns
```

Out[52]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

In [53]:
```
churn_rate = df['churn_flag'].mean()*100
print('churn Rate = ',round(churn_rate,2),"%")
```

```
churn Rate =  28.57 %
```

In [54]:
```
#2 - Retaintion rate
retaintion_rate = 100 - churn_rate
print("Retention Rate = ",round(retaintion_rate,2),"%")
```

```
Retention Rate =  71.43 %
```

In [55]:
```
df.head(2)
```

Out[55]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>...</th> <th>churn_flag</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> </thead> <tbody> <th>0</th> <th>1</th> </tbody> <tbody></tbody>

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	cltv	...	churn_flag	customer_name	country	state	gender	dob	complaint_date	escalations	csat_score	complaint_count
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	0	kesl								
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	1	ragl								

2 rows × 21 columns

In [56]:
```
#3 churn by plan type
```

In [57]:
```
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).res
print(churn_by_plan)
```

```
  plan_type  churn_rate_pct
0     Basic           60.00
1   Premium           14.29
2  Standard           22.22
```

In [58]:
```
#4 churn by state
# and churn by sub
```

In [59]:
```
#5 ARPU - REVENU per user
```

In [60]:
```
arpu = df['monthly_charges'].mean()
print('ARPU = ',round(arpu,2))
```

```
ARPU =  18.85
```

In [61]:
```
#6 Avg customer tenure
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),

    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)

df.head()
```

Out[61]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>...</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> <th>tenure_days</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	kesha
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	ragha
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	...	lalit
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	...	moha
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	...	mir

5 rows × 22 columns

In [62]:
```
avg_tenure = df['tenure_days'].mean()
print('Avg Tenure (Days) =',round(avg_tenure),0)
```

```
Avg Tenure (Days) = 1570 0
```

In [63]:
```
#7 revenu at risk - revenu last from churn user
revenu_at_risk = df.loc[df['churn_flag']==1,'monthly_charges'].sum()
print('Revenu at risk (Rs K) =', revenu_at_risk)
```

```
Revenu at risk (Rs K) = 73.94
```

In [72]:
```
#8 Esclation rate
escalation_rate = (df['escalations']== 'y').mean()*100
print("Esclation Rate = ", round(escalation_rate,2),"%")
```

```
Esclation Rate =  0.0 %
```

In [73]:
```
#9 Avg complaint late
avg_complaint = df['complaint_count'].sum() / df['customerid'].nunique()
print("AVG complaint = ",round(avg_complaint,2))
```

```
AVG complaint =  0.43
```

In [76]:
```
#10 - correlation esclation vschurn
```

```
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(
    corr_df['churn_flag']
)
print("correlation between escalation vs churn is: ", round(correlation,2))
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[76], line 5
      1 #10 - correlation esclation vschurn
      2
      3 corr_df = df[['escalations', 'churn_flag']].dropna()
      4
----> 5 correlation = corr_df['escalations'].corr(
      6     corr_df['churn_flag']
      7 )
      8 print("correlation between escalation vs churn is: ", round(correlation,2))

File ~\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\series.
py:2770, in Series.corr(self, other, method, min_periods)
   2767 if len(this) == 0:
   2768     return np.nan
-> 2770 this_values = this.to_numpy(dtype=float, na_value=np.nan, copy=False)
   2771 other_values = other.to_numpy(dtype=float, na_value=np.nan, copy=False)
   2773 if method in ["pearson", "spearman", "kendall"] or callable(method):

File ~\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\base.p
y:664, in IndexOpsMixin.to_numpy(self, dtype, copy, na_value, **kwargs)
    574 """
    575 A NumPy ndarray representing the values in this Series or Index.
    576
   (...)    661       dtype='datetime64[ns]')
    662 """
    663 if isinstance(self.dtype, ExtensionDtype):
--> 664     return self.array.to_numpy(dtype, copy=copy, na_value=na_value, **kwarg
s)
    665 elif kwargs:
    666     bad_keys = next(iter(kwargs.keys()))

File ~\AppData\Local\Programs\Python\Python314\Lib\site-packages\pandas\core\arrays
\arrow\array.py:1746, in ArrowExtensionArray.to_numpy(self, dtype, copy, na_value)
   1744 result = data._pa_array.to_numpy()
   1745 if dtype is not None:
-> 1746     result = result.astype(dtype, copy=False)
   1747 if copy:
   1748     result = result.copy()

ValueError: could not convert string to float: 'Y'
```

In [77]:

```
#11 - churn risk count
condition = [
    (df['churn_score'] < 50),
    (df['churn_score'] <= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]
```

```
choices = ['low','med','high']
df['churn_risk'] = np.select(condition,choices,default="unkown")
```

In [78]:
```
df.columns
```

Out[78]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk'],
      dtype='str')
```

In [ ]:
```

```

In [79]:
```
df.head(2)
```

Out[79]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>...</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> <th>tenure_days</th> <th>churn_risk</th> </thead> <tbody> <th>0</th> <th>1</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	India										
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	India										

2 rows × 23 columns

In [ ]:
```

```

In [80]:
```
# Visualization using matplotlib and seaborn
```

In [81]:
```
df.head()
```

Out[81]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>...</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> <th>tenure_days</th> <th>churn_risk</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	cltv	...	country	state	gender	dob	complaint_date	escalations	csat_score	complaint_count	tenure_days	churn_risk
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	India									
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	India									
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	...	India									
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	...	India									
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	...	India									

5 rows × 23 columns

In [82]:
```
# copy original df
df_visual = df.copy()
```

In [83]:
```
df_visual.columns
```

Out[83]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk'],
      dtype='str')
```

In [84]:
```
# Monthly churn trend (time seies kpi)
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')

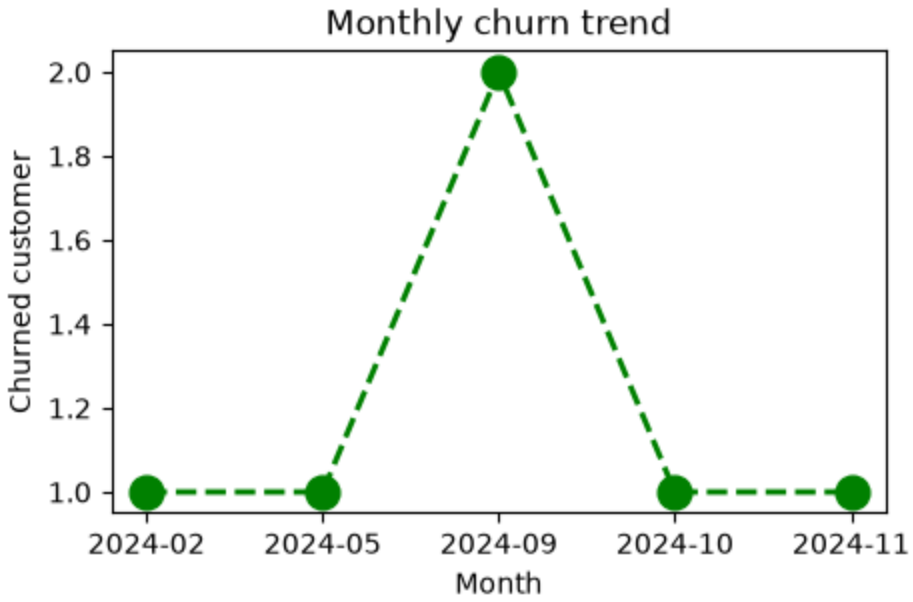
churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month')
```

In [85]:
```
churn_trend.head()
```

Out[85]:
```
cancellation_month
2024-02    1
2024-05    1
2024-09    2
2024-10    1
2024-11    1
Freq: M, dtype: int64
```

In [86]:
```
plt.figure(figsize =(5,3))
plt.plot(churn_trend.index.astype(str),churn_trend.values,color='green', marker='o'

plt.title('Monthly churn trend')
plt.xlabel('Month')
plt.ylabel('Churned customer')
plt.show()
```



In [87]:
```
#churn by plan type
df_visual.head()
```

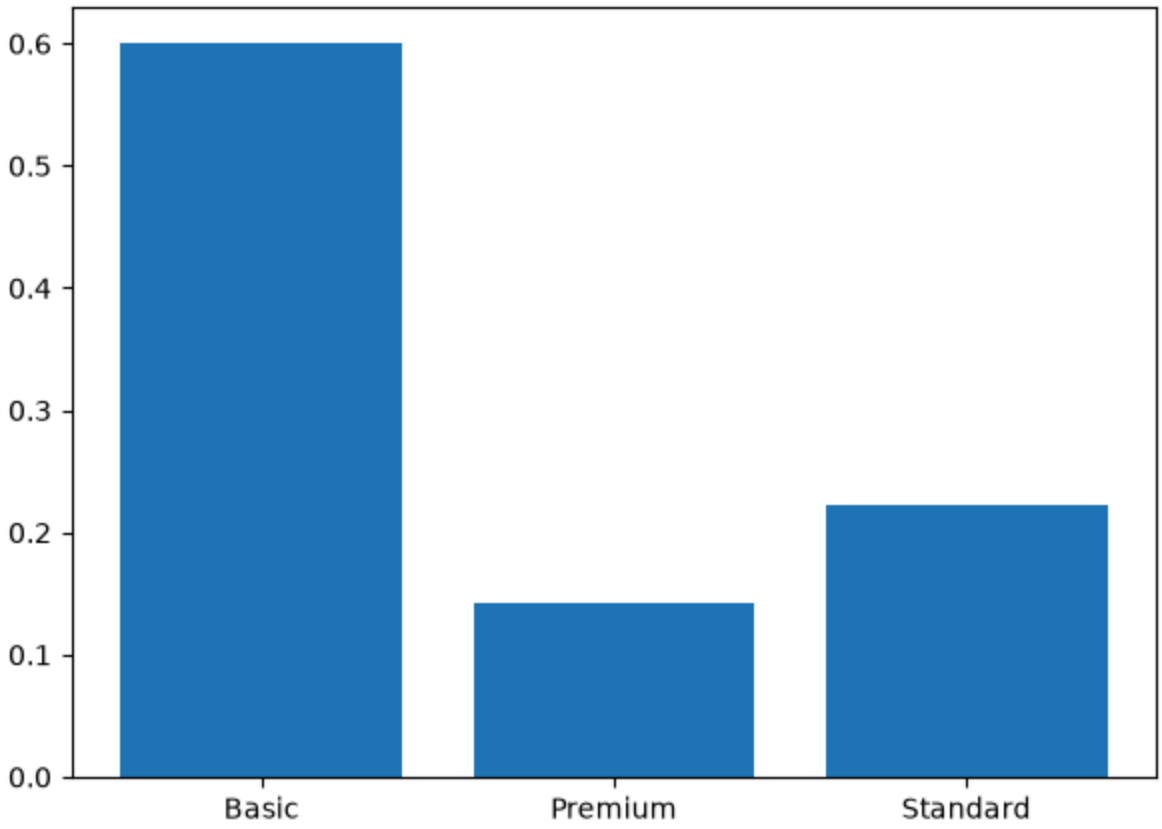
Out[87]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>...</th> <th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> <th>tenure_days</th> <th>churn_risk</th> <th>cancellation_month</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	Maha
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	Kar
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	...	
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	...	Na
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	...	

5 rows × 24 columns

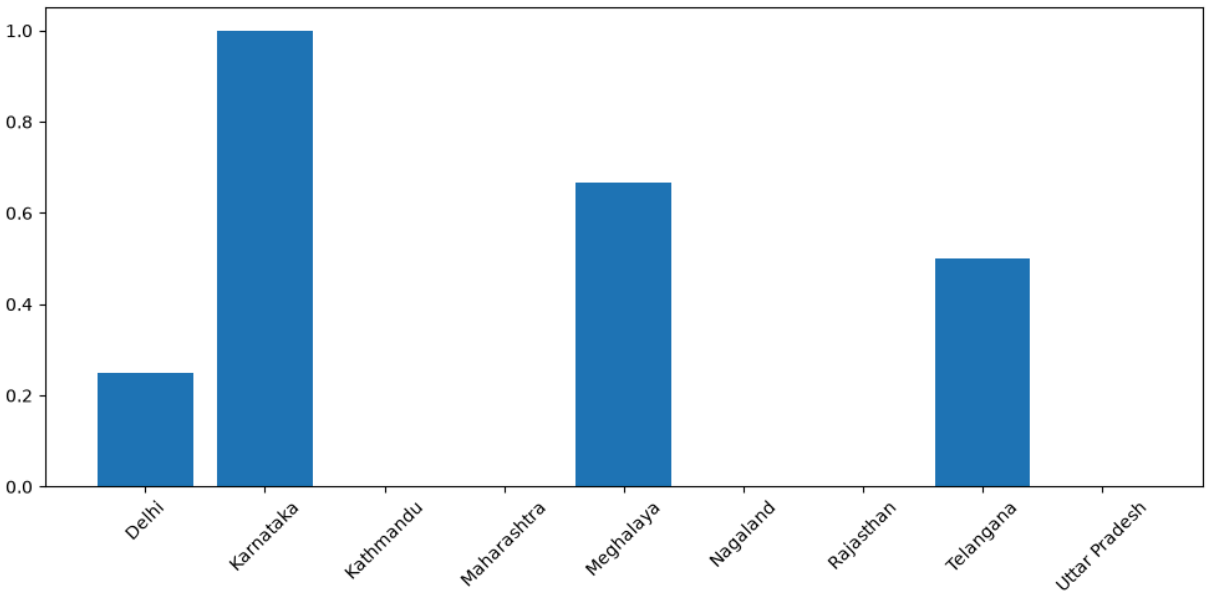
In [88]:
```
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

plt.figure(figsize = (7,5))
plt.bar(churn_plan.index, churn_plan.values)
plt.show()
```



In [89]:
```
# churn by states
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

plt.figure(figsize = (12,5))
plt.bar(churn_plan.index, churn_plan.values)
plt.xticks(rotation=45)
plt.show()
```



In [90]:
```
#encoding
df_visual.columns
```

Out[90]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [91]:
```
df_visual[['plan_type', 'contract_type','churn_score','churn_flag','escalations']].
```

Out[91]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn_flag</th> <th>escalations</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

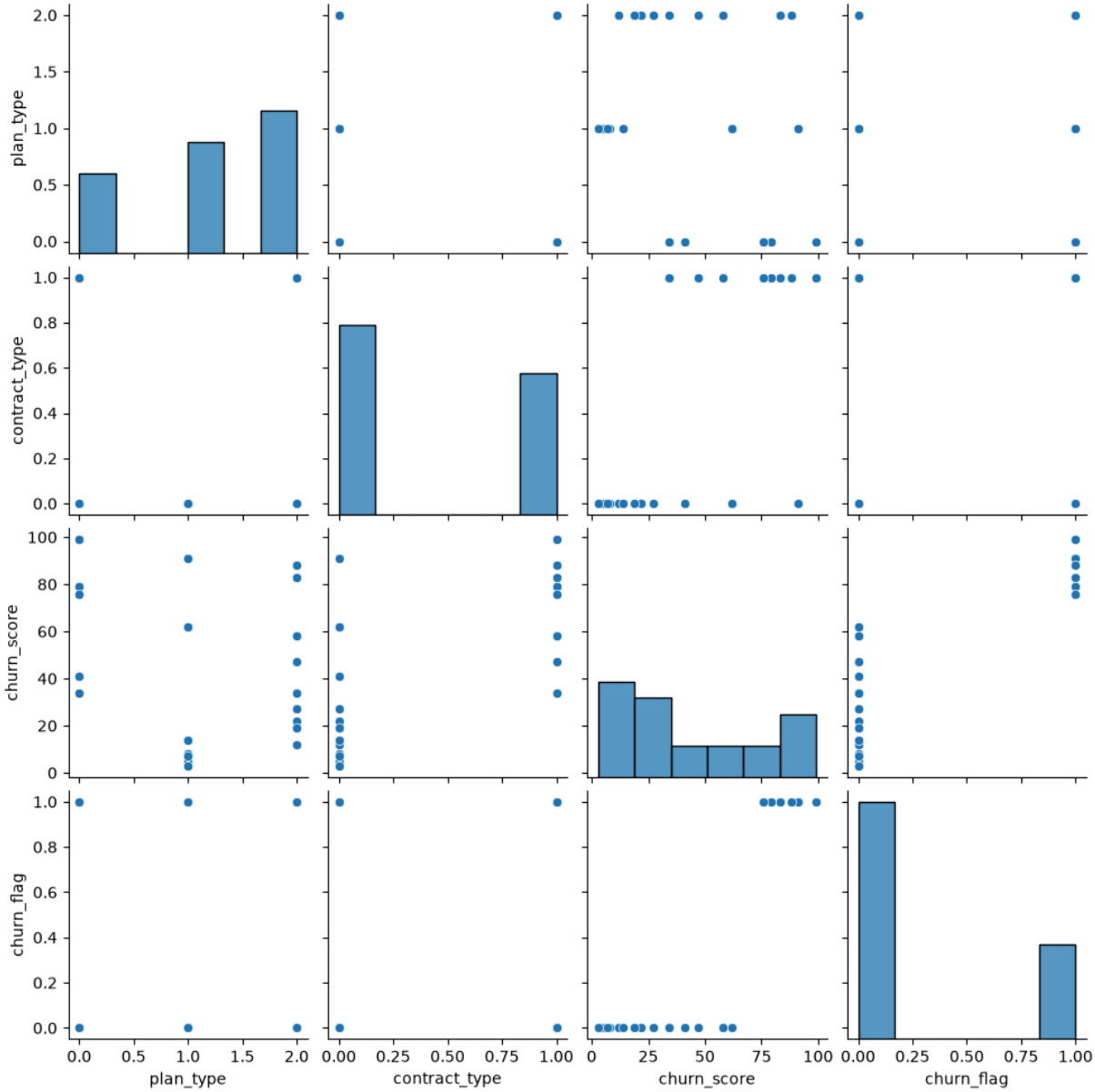
Standard	Annual	12	0	NaN
Premium	Annual	91	1	Y
Basic	Monthly	34	0	NaN
Premium	Annual	8	0	NaN
Standard	Monthly	88	1	Y

In [ ]:
```
```

In [92]:
```
df_encoded = df_visual[['plan_type', 'contract_type','churn_score','churn_flag','es
categorical_cols = ['plan_type','contract_type']
for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

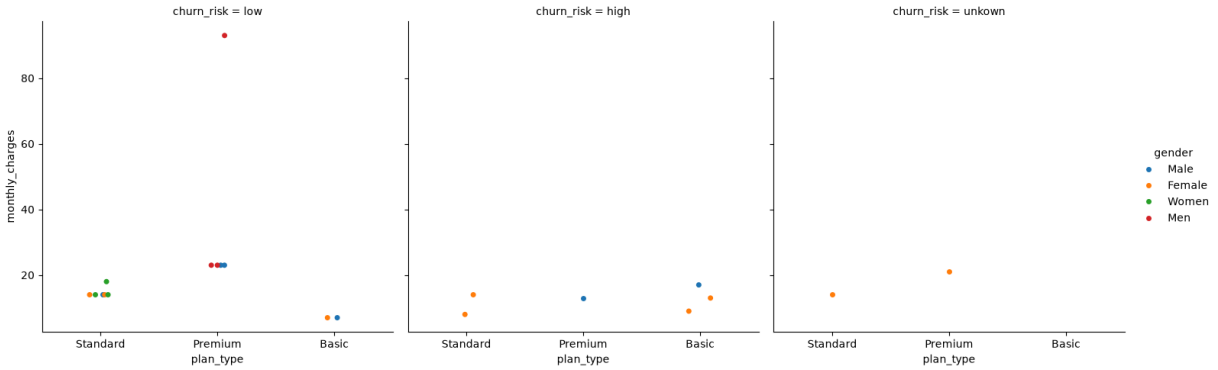
In [93]:
```
#pair plot
sns.pairplot(df_encoded)
```

Out[93]:
```
<seaborn.axisgrid.PairGrid at 0x2690586ea50>
```



In [96]:
```
#catplot/facgrid
plt.figure(figsize=(8,5))
sns.catplot(data = df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_risk',
            )
```

Out[96]:
```
<seaborn.axisgrid.FacetGrid at 0x2690a33b9d0>
<Figure size 800x500 with 0 Axes>
```



In [97]:
```
#pivt tavble
```

In [100…
```
pd.pivot_table(data=df_visual,
               values='churn_flag',
               index='plan_type',
               aggfunc='mean').reset_index()
```

Out[100…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>plan_type</th> <th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

	plan_type	churn_flag
0	Basic	0.600000
1	Premium	0.142857
2	Standard	0.222222

In [106…
```
pd.pivot_table(
                 df_visual,
               values=['monthly_charges','customerid','churn_flag'],
               index='plan_type',
               aggfunc={
                   'monthly_charges':'sum',
                   'customerid':'nunique',
                   'churn_flag':'mean'
               })
```

Out[106…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th> <th>plan_type</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Basic</th> <th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

	churn_flag	customerid	monthly_charges
plan_type			
Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91

In [107…
```
#workoing with sql in python(pandas)
```

In [109…
```
#create db in sql
conn = sqlite3.connect('test_database.sqlite')


#table details
conn.execute("CREATE TABLE users(first_name TEXT,country TEXT,budgetm INTEGER)")

conn.commit()
```

In [111…
```
#insert data

cursor = conn.cursor()
cursor.execute(
    """
       INSERT INTO users VALUES
       ("tejas",'india',50000),
       ('prince','japan',40000),
       ('ranvijay','nepal',30000)

       """
)

conn.commit()

print("Work done")
```

```
Work done
```

In [114…
```
#check insert data in table
conn = sqlite3.connect('test_database.sqlite')
query = """SELECT * FROM users"""
df_results = pd.read_sql(query,conn)
df_results.head()
```

Out[114…
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>first_name</th> <th>country</th> <th>budgetm</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

	tejas	india	50000
	prince	japan	40000
	ranvijay	nepal	30000

In [116…
```
#aggregation
query = """
SELECT country , sum(budgetm) as total_budget
FROM users
GROUP BY country
"""
df_agg = pd.read_sql(query, conn)
df_agg
```

Out[116…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th> </th> <th>country</th> <th>total_budget</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

india	50000	
japan	40000	
nepal	30000	

In [117…

```
conn.close()
```

In [ ]: